Week 1 Write Up for

Master of Science

Information and Communications Technology

Melissa Knapp

University of Denver University College

April 5, 2026

Faculty: Nathan Braun, MA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Table of Contents

Overview

The overall problem being solved is how to track the visitors to various parking lots and charge them for time parked each month. The system, among other functions, registers customers, their cars, and tracks where and when they park. This week we created a layer in the system that would handle registering cars and customers.

I decided to start from the Netbeans code base linked in the week 1 assignment. I was grateful to have something fresh to start with as I was worried that I might have bugs and artifacts that I never took care of in my initial version.

I first started by creating shells for the new interface and classes: “Command”, “RegisterCarCommand”, and “RegisterCustomerCommand”. It took me some time to understand exactly what I needed to do with these classes. The diagrams were helpful in showing how they interact.

Challenges Faced

The biggest challenge for me, by far, was that it has been a while since I’ve done any coding in Java. I took Object Oriented Methods & Programming 1 in the winter quarter of 2025, so I’m having to remind myself of the syntax and notation. And I had forgotten how to update versions and download dependencies to my local machine so it took a few tries before the code would compile without errors. I used JDK 25 and Junit version 5.8.2.

Another major challenge I faced this week was trying to familiarize myself with someone else's code. It was well commented, which unfortunately is something I struggle with myself, so that did help some. However, the biggest difficulty for me was the Address class. It used Builders, which I had never encountered before. So, I spent some time reading up on them to

see how they work. I have a basic grasp now of how to create a Builder instance, call the various functions to set each attribute and then call the build function.

I initially didn't create the logic for building addresses in the RegisterCustomerCommand class because I was unsure of how the addresses would be passed through in the Properties object. I understand the "key": "value" aspect of how it works, that each key-value pair would be passed in as strings, but passing in a full address as a string would make it hard to tokenize it to store as separate attributes in an Address class. At which point it occurred to me that each section of the address could be passed in as its own key-value pair, rather than using a single key-value for the entire address. But that did leave me with the problem of parameter checking as it was unclear which parts of the address would be required, and which wouldn't. Obviously, street address 2 would be optional, but would the state or zip be required? I decided to code the logic with the assumption that everything but the street address 2 is required, as that seemed to make the most sense to me.

JUnit Testing

For the unit testing for both Command classes, I focused mainly on the checkParameters function, as I felt that would be where any bugs would be found. For the registerCustomerCommand I ensured that both my clean-up of the phone number (I removed any spaces, dashes, and parentheses) worked correctly by setting a valid phone number with extra spaces in it. And I tested that my logic for phone number length worked as well by setting the phone number to a 10-digit number starting with 1 for the country code and 9 digits as both these numbers would be a digit short. I also only tested the exception for a blank zip code

parameter; I felt that since all the parameters used the same process for checking empty values it would be excessive to write the code testing all 7 key-value pairs. For the execute command I wasn't sure how the ID maker worked so I cheated a bit by letting the test fail initially as I knew it would tell me what the ID should be and then I set that to the expected response string.

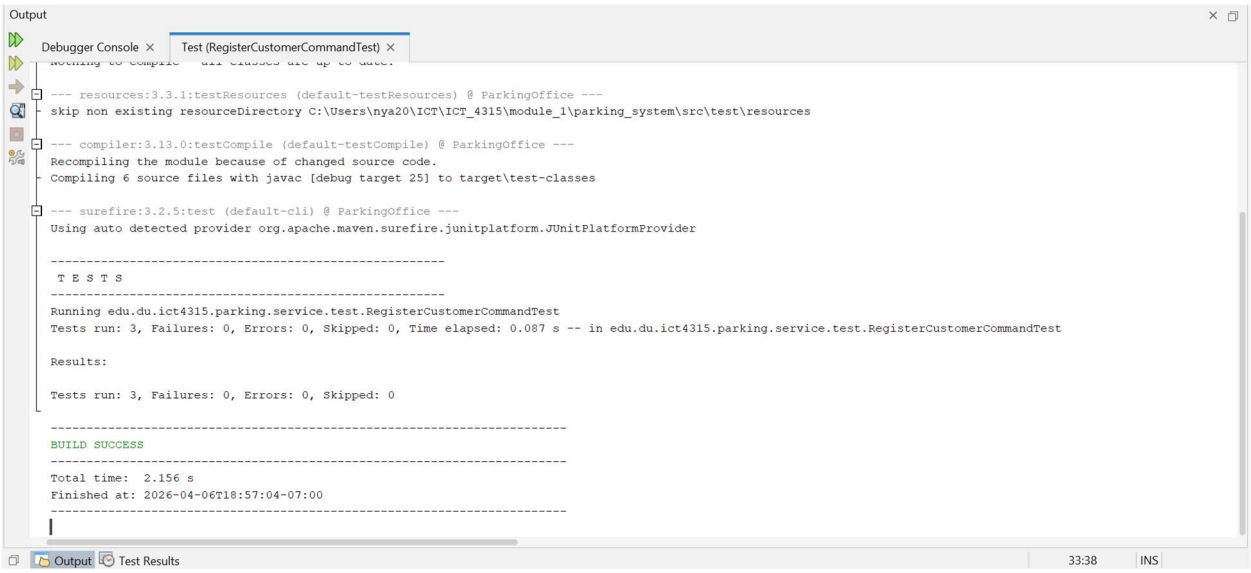

Figure 1. JUnit test results for RegisterCustomerCommandTest

For the RegisterCarCommand testing, I tested that setting the car type to something other than SUV or Compact would throw an exception and that having an empty value would also throw an exception. I followed the same process for the execute function by letting it fail to see what the permit number would be and then setting the expected value to that.

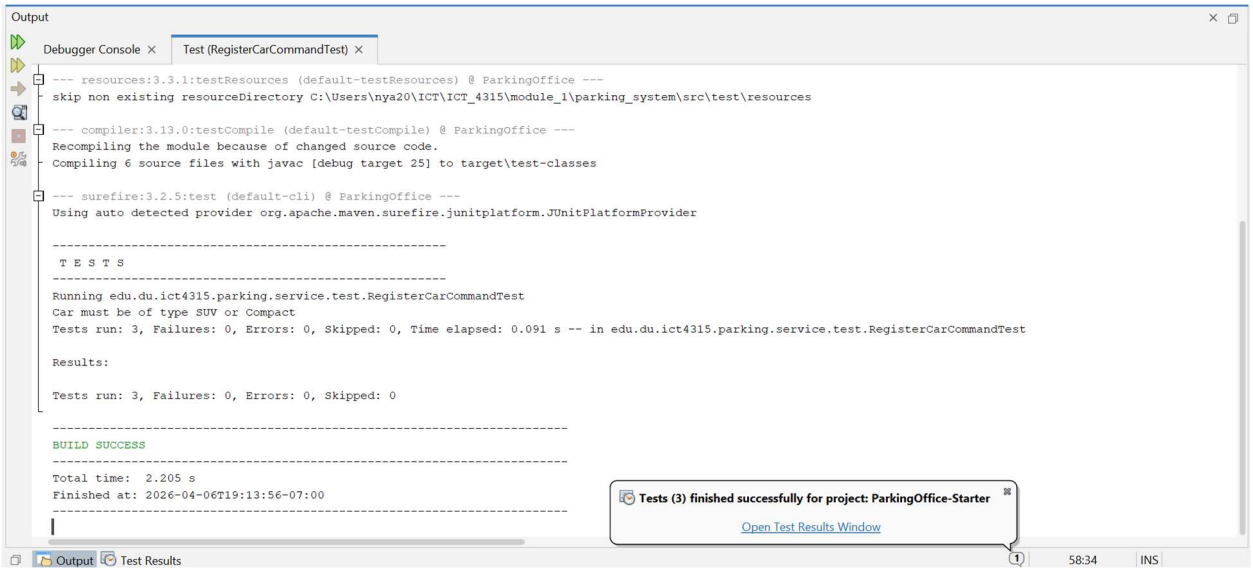

Figure 2. JUnit test results for RegisterCarCommandTest

Future Considerations

One of the things I plan on focusing on in the coming weeks is learning how to use the Logger library. I am unfamiliar with that particular library and used Exceptions to handle any problems with my parameter validation. I don't know if there are any specific advantages of using logs vs exceptions, but I do want the entire code base to have a consistent way of handling problems and logging is already implemented in many of the classes.

References